UTM
UNIVERSITI TEKNOLOGI MALAYSIA
Department of Computer Science
Faculty of Computing

SECP3623 - Database Programming

(*Project 2*)

Programme	:	Bachelor of Computer Science (Data *Engineering*)
Subject Code	:	SECP 3623
Subject Name	:	Database Programming
Session-Sem	:	2025/2026-01

Prepared by :
1) Muhammad Afiq Danish bin Mohd Hazni (A23CS0118)
2) Muhammad Adam bin Razali (A23CS0116)
3) Dheshieghan A/L Saravana Moorthy (A23CS0072)
4) Pravinraj A/L Sivabathi (A23CS0171)

Section : 02

Lecturer : Dr. Shamini A/P Raja Kumaran

1. Introduction

1.1 Domain Description

Event Management System

The domain that we chose for this project is an Event Management System that is designed to manage scheduled events and activities. This system focuses on a NoSQL database implementation using MongoDB to handle workload including participants registrations, event scheduling, as well as session management. The MongoDB document model allow for flexibility of storing complex data containing lists and dates without constraints of a traditional database schema.

1.2 Project Objectives

The main objectives of this project include:

- **NoSQL application** - developing document data model for a real-world scenario
- **CRUD Implementation** - Implementing operations by inserting, finding, updating, and deleting documents within MongoDB collections.
- **Performance Optimization** - Creation of single-field and compound indexes.
- **Advanced SQL Operations** - Build aggregation pipelines to transform data and generate analytical insights, such as total revenue and registration counts.
- **Security Analysis** - Analyze security considerations and design challenges inherent in NoSQL systems.

1.3 Team Members & Roles

Name	Role
Dheshieghan A/L Saravana Moorthy	Designed the collection structures and implemented indexing strategies.
Muhammad Adam bin Razali	Developed CRUD operations and verified query logic with projections.
Pravinraj A/L Sivabathi	Built aggregation pipelines for financial and registration reporting.
Muhammad Afiq Danish bin Mohd Hazni	Formulated the project report and conducted security analysis.

2. NoSQL Data Model

2.1 List and Explanation of Collections

2.1.1 Users Collection (users)

The users collection stores information related to all users in the system, including event attendees and event organizers. Each user has a unique identifier and may participate in multiple events or organize several events. By storing user information in a separate collection, the system avoids duplicating user data across multiple event registrations. This design ensures better data consistency, as any update to user information (such as phone number or email) only needs to be made in one place.

2.1.2 Events Collection (events)

The events collection stores core information about events such as the event title, location, date, and ticket options. Each event is associated with an organizer through a reference to the users collection. Ticket types are embedded within the event document because they are tightly coupled with the event and are usually retrieved together. This structure improves read performance when displaying event details and available ticket options to users.

2.1.3 Registrations Collection (registrations)

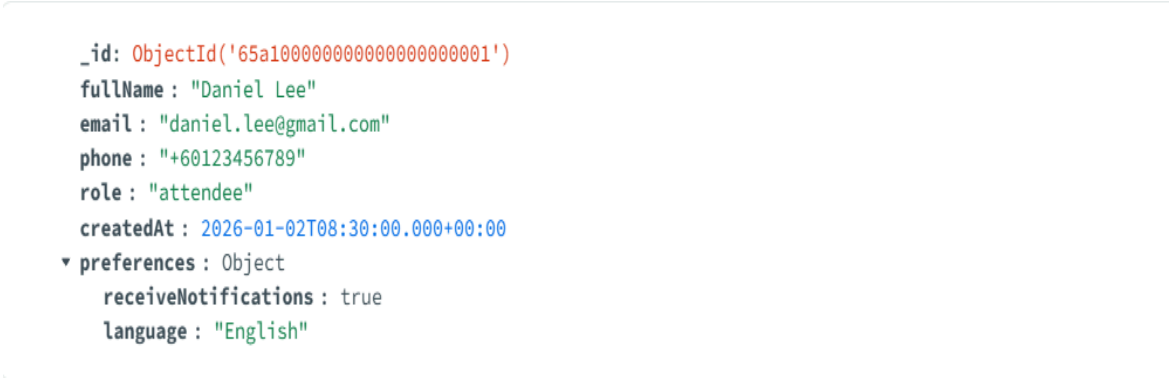
The registrations collection records each event registration made by users. It acts as a linking collection between users and events, representing a many-to-many relationship. In addition to references to the user and event, this collection stores transactional information such as ticket details, payment status, and registration time. Embedding payment and ticket data ensures that historical registration records remain accurate even if event prices change in the future.

2.1.4 Sessions Collection (sessions)

The sessions collection stores information about individual sessions or agendas within an event. An event may consist of multiple sessions, each with different speakers, time slots, and locations. Each session document references its parent event while embedding speaker details. This approach allows efficient retrieval of session data without requiring additional joins for frequently accessed speaker information.

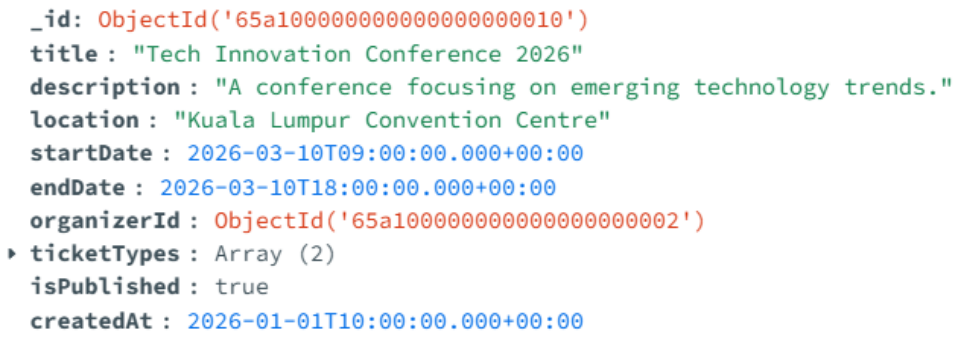
2.2 Example JSON Documents

2.2.1 Users Collection



This document demonstrates how MongoDB supports nested objects and flexible schemas by embedding user preferences directly within the user document.

2.2.2 Events Collection



This document shows how arrays of embedded documents are used to store ticket types, allowing flexible pricing and quota management.

2.2.3 Registrations Collection

```
_id: ObjectId('65a1000000000000000100')
registrationCode : "EVT-2026-0045"
userId : ObjectId('65a1000000000000000001')
eventId : ObjectId('65a1000000000000000010')
▸ ticket : Object
▸ payment : Object
status : "CONFIRMED"
registeredAt : 2026-02-15T06:40:00.000+00:00
```

This document highlights the use of embedded payment data and references to maintain both data accuracy and integrity.

2.2.4 Sessions Collection

```
_id: ObjectId('65a1000000000000000200')
eventId : ObjectId('65a1000000000000000010')
title : "AI in Everyday Applications"
▸ speaker : Object
startTime : 2026-03-10T10:00:00.000+00:00
endTime : 2026-03-10T11:30:00.000+00:00
room : "Hall A"
```

This document illustrates embedding speaker information for efficient session display and management.

2.3 Explanation of Embedding vs Referencing

Embedding is used when data is closely related and commonly accessed together, such as ticket types within an event or payment details within a registration. This approach reduces the need for additional queries and improves read performance.

Referencing is applied when data is shared across multiple entities or may change independently, such as user profiles and event details. By referencing these entities, the system avoids unnecessary data duplication and ensures consistency across collections.

The combination of embedding and referencing allows the system to balance performance, storage efficiency, and maintainability.

2.4 Data Types Used

Data Type	Usage Example
ObjectId	_id, userId, eventId
String	names, email, role, status
Number	ticket price, quota
Boolean	isPublished, notification preferences
Date	event dates, payment date
Object	embedded documents (payment, speaker)
Array	ticket types list

3.0 CRUD Implementation Summary

- **Insert operation**

```
> db.users.insertMany([
  { "_id": "user_01", "name": "Muhammad Adam", "email": "adam@gmail.com", "role": "organizer", "city": "Johor Bahru", "phone": "012-3456789" },
  { "_id": "user_02", "name": "Afiq", "email": "afiq@gmail.com", "role": "organizer", "city": "Kuala Lumpur", "phone": "019-8765432" },
  { "_id": "user_03", "name": "Dhesh", "email": "dhesh@gmail.com", "role": "organizer", "city": "Penang", "phone": "017-4455667" },
  { "_id": "user_04", "name": "Pravinraj", "email": "pravinraj@gmail.com", "role": "attendee", "city": "Kuala Lumpur", "phone": "016-2233445" },
  { "_id": "user_05", "name": "Sarah Lee", "email": "sarah.lee@gmail.com", "role": "organizer", "city": "Cyberjaya", "phone": "013-9988776" },
  { "_id": "user_06", "name": "Ali Ahsan", "email": "ali.ahsan@gmail.com", "role": "attendee", "city": "Shah Alam", "phone": "014-5566778" },
  { "_id": "user_07", "name": "Jessica Wong", "email": "jess.wong@gmail.com", "role": "attendee", "city": "Ipoh", "phone": "011-1122334" },
  { "_id": "user_08", "name": "Siti Aminah", "email": "siti.aminah@gmail.com", "role": "organizer", "city": "Melaka", "phone": "018-7766554" },
  { "_id": "user_09", "name": "Farid Kamil", "email": "farid.kamil@gmail.com", "role": "attendee", "city": "Johor Bahru", "phone": "019-3344556" },
  { "_id": "user_10", "name": "Nurul Izzah", "email": "nurul.izzah@gmail.com", "role": "attendee", "city": "Kuala Lumpur", "phone": "012-9988112" }
]);
```

```
{
  acknowledged: true,
  insertedIds: {
    '0': 'user_01',
    '1': 'user_02',
    '2': 'user_03',
    '3': 'user_04',
    '4': 'user_05',
    '5': 'user_06',
    '6': 'user_07',
    '7': 'user_08',
    '8': 'user_09',
    '9': 'user_10'
  }
}
```

To expand the number of data inside each collection efficiently, the insertMany() command was utilized across the collections until the data reached 10 for each collection. In the events collection, the ticket information was embedded directly into each document due to the advantage of the NoSQL document model. It makes it easier to retrieve the associated ticked attributes without making a secondary query. Similarly, the registrations collection also embeds their payment details while maintaining the references to the users and events collection through user_id, and event_id therefore maintaining balance between the data.

- **query operation**

The query operation was demonstrated through the use of find() method which allows the user to filter and retrieve specific data.

On the left side of the image above, we used basic filtering to retrieve all of the events happening in “Dataran Merdeka” by passing in simple key-value pairs. The system then performs matching on the string fields to produce the output.

To query the financial data, the $and logic as well as $lt operator was used to find the registrations that are already “PAID” while also having a payment amount of less that RM 50, therefore demonstrating simple analytics to find specific types of users.

- **update operation**

The update operations were designed to handle the administrative tasks as well as transactional data changes using specific operators. The use of the $set operator with UpdateOne() method as displayed in the image above was able to change a specific user's phone number.

Furthermore, update operation can also involve array modification through the use of $push operator. For example, as seen in the leftmost image above, the operator was used to append a new "cat" to a session's category list. This shows the capability of MongoDb to extend the document model structures dynamically instead of altering a predefined schema.

The update operation is also capable of atomic counters to manage ticket inventory through the use of the $inc operator. the ticket will be incremented and decremented whenever there are transactions of ticket quantities to ensure the data accuracy during concurrent booking requests.

- **delete operation**

the delete operations demonstrate the implementation of removing specific documents as shown above. Single deletion using the deleteOne() method removes specific document in this case, a session containing the id of “ses_10”. The batch deletion can also be done to increase efficiency through the use of deleteMany() command such as removing all registrations that have been marked as "CANCELLED" to free up storage space and maintain database hygiene.

Projections and Query Tests

Query Test - The Query Test confirms that the database can accurately filter data based on specific conditions. For instance, by searching for "Kuala Lumpur" events with "VIP" tickets priced under RM 100, we demonstrate that the system can handle complex logic involving both

top-level fields and nested arrays. This proves the database is reliable for finding precise information within a large dataset.

Projection - projection as in the query above is used to control which fields are visible in the search results to keep the output clean. By explicitly including only the title, location, and ticketTypes while hiding the technical _id field, we ensure the data is easy to read and relevant to the user. This optimization prevents the system from displaying unnecessary internal codes or loading extra data.

4.0 Advanced MongoDB Operations

4.1 Indexing

Indexing is an essential technique in MongoDB that improves query performance by allowing the database to quickly locate required documents without scanning the entire collection. In an Event Management System, queries are frequently executed to search for events by date and to retrieve registrations based on event and payment status. As the number of events and registrations increases, performing full collection scans would significantly reduce system efficiency. Therefore, indexing is implemented to ensure faster query execution, better scalability, and improved overall database performance.

4.1.1 Single Field Index

```
> db.events.createIndex({ date: 1 })
< date_1
```

A single-field index was created on the date field in the events.This index improves the performance of queries that filter or sort events by their scheduled date. Since users commonly view upcoming or past events in chronological order, indexing the date field allows MongoDB to retrieve and sort event documents efficiently. Without this index, MongoDB would need to scan every event document to determine its date, which would be inefficient as the dataset grows. This index ensures that event-related queries remain fast and responsive.

4.1.2 Compound Index

```
> db.registrations.createIndex({ event_id: 1, status: 1 })
< event_id_1_status_1
```

A compound index was created on the event_id and status fields in the registrations collection. This compound index is designed to optimize queries that filter registrations based on both the event and the registration status. In real-world usage, administrators frequently retrieve paid or pending registrations for a specific event. By indexing both fields together, MongoDB can efficiently locate only the relevant documents without scanning unrelated registration records. This significantly improves query performance for multi-condition queries and supports efficient registration management.

4.2 Aggregation Pipelines

Aggregation pipelines in MongoDB are used to process, transform, and analyze data in stages. Each stage performs a specific operation, such as filtering, grouping, or calculating totals. In the Event Management System, aggregation pipelines are essential for generating analytical insights, such as counting registrations, calculating revenue, and summarizing registration statuses. These operations help administrators make data-driven decisions and understand event performance more effectively.

4.2.1 Total Registrations per Event

```
> db.registrations.aggregate([
    {
      $group: {
        _id: "$event_id",
        totalRegistrations: { $sum: 1 }
      }
    }
  ])
< {
    _id: 'evt_04',
    totalRegistrations: 2
  }
  {
    _id: 'evt_03',
    totalRegistrations: 1
  }
  {
    _id: 'evt_01',
    totalRegistrations: 2
  }
  {
    _id: 'evt_02',
    totalRegistrations: 2
  }
  {
    _id: 'evt_05',
    totalRegistrations: 1
  }
  {
    _id: 'evt_08',
    totalRegistrations: 1
  }
  {
    _id: 'evt_10',
    totalRegistrations: 1
  }
```

The following aggregation pipeline was used to calculate the total number of registrations for each event. This aggregation groups all registration documents by their event_id and counts the number of registrations associated with each event. The result provides a clear summary of event popularity by showing how many users registered for each event. This information is useful for evaluating audience interest and planning future events.

4.2.2 Total Revenue per Event

```
> db.registrations.aggregate([
    { $match: { status: "PAID" } },
    {
      $group: {
            _id: "$event_id",
            totalRevenue: { $sum: "$payment.amount" }
      }
    }
  ])
< {
    _id: 'evt_08',
    totalRevenue: 0
  }
  {
    _id: 'evt_04',
    totalRevenue: 80
  }
  {
    _id: 'evt_01',
    totalRevenue: 650
  }
  {
    _id: 'evt_02',
    totalRevenue: 40
  }
  {
    _id: 'evt_10',
    totalRevenue: 100
  }
```

The following aggregation pipeline was used to calculate total revenue generated by each event. This pipeline first filters the registration records to include only those with a PAID status, ensuring that only completed payments are considered. It then groups the filtered records by event_id and calculates the total revenue by summing the payment amounts. This aggregation provides financial insights into each event and helps organizers evaluate which events generate the highest revenue.

4.2.3 Registration Status Summary

```
> db.registrations.aggregate([
    {
      $group: {
      _id: "$status",
      count: { $sum: 1 }
      }
    }
  ])
< {
    _id: 'PENDING',
    count: 2
  }
  {
    _id: 'PAID',
    count: 7
  }
  {
    _id: 'CANCELLED',
    count: 1
  }
```

The following aggregation pipeline was used to summarize registrations based on their status. This aggregation groups registration records by their status, such as PAID, PENDING, or CANCELLED, and counts the number of records in each category. The resulting summary allows administrators to monitor payment completion rates and identify outstanding registrations that require follow-up action.

4.3 Sorting

Sorting is used to arrange query results in a meaningful order, making the output easier to understand and analyze. In an Event Management System, sorting is particularly useful for displaying events in chronological order and identifying high-value registrations. Sorting enhances data presentation and supports better decision-making.

4.3.1 Sorting Events by Date (Ascending)

```
> db.events.find().sort({ date: 1 })
< {
    _id: 'evt_03',
    title: 'AI Workshop',
    date: 2026-01-20T00:00:00.000Z,
    location: 'UTM KL',
    organizer_id: 'user_03',
    status: 'completed',
    tickets: [
      {
        type: 'Student',
        price: 50,
        qty: 30
      },
      {
        type: 'Professional',
        price: 100,
        qty: 20
      }
    ]
  }
```

The following query sorts events by date in ascending order. This sorting operation arranges events from the earliest to the latest date. It is especially useful for displaying upcoming events in a logical sequence, allowing users to easily plan their participation.

4.3.2 Sorting Events by Date (Descending)

```
> db.events.find().sort({ date: -1 })
< {
    _id: 'evt_09',
    title: 'Durian Fiesta',
    date: 2026-08-15T00:00:00.000Z,
    location: 'Raub',
    organizer_id: 'user_08',
    status: 'planned',
    tickets: [
      {
        type: 'Buffet',
        price: 80,
        qty: 150
      }
    ]
  }
```

The following query sorts events by date in descending order. This operation displays the most recent or latest events first. It is useful for administrative review and reporting, where recent events are often prioritized.

4.3.3 Sorting Registrations by Payment Amount

```
> db.registrations.find().sort({ "payment.amount": -1 })
< {
    _id: 'reg_02',
    user_id: 'user_06',
    event_id: 'evt_01',
    reg_date: 2026-01-10T06:48:51.814Z,
    status: 'PAID',
    payment: {
      method: 'FPX',
      amount: 500
    }
  }
```

The following query sorts registrations based on payment amount in descending order. This sorting operation helps identify registrations with the highest payment values. It is useful for

financial analysis and for understanding which registrations contribute most significantly to event revenue.

5.0 Security & Challenges

5.1 Security Concerns in NoSQL Systems

In an event management system, sensitive records such as user profiles, event details, registrations and payment information are stored in MongoDB collections. NoSQL databases are commonly accessed through web APIs, making them vulnerable if proper security measures are not implemented.

Without a proper protection mechanism, the system may face risks such as unauthorized access, data leakage or accidental modification and deletion of records. Since MongoDB is schema-less, incorrect or malicious data structures may also be inserted if validation is not enforced. This can lead to inconsistent data and potential system misuse. Therefore, security must be considered at both the database and application levels to protect user information and maintain data integrity.

5.2 Access Control and Injection Risks

MongoDB supports role-based access control (RBAC) which means allowing different permissions to be assigned to different user roles. In a real-world event management system, roles such as admin, organizer and user should be clearly separated.

For example:

1. Admin manages all data and system settings
2. Organizers create and update events
3. Users only view events and register.

Applying this principle ensures that each role can only perform the actions required for its function and reduce the risk of misuse.

Injection risk occurs when user input is directly used in database queries. In NoSQL systems, the attackers may exploit query operators such as $or or $ne to bypass filters which is a technique known as NoSQL injection. This can allow unauthorized access to restricted data. To mitigate this risk, the system should validate and sanitize all user inputs, avoid using raw objects in queries and enforce fixed query structures in the application logic.

5.3 Consistency Trade-offs in Distributed Systems

MongoDB is designed for scalability and high availability. It often operates in distributed environments such as replica sets. This introduces a trade-off between consistency and availability. In some configurations, MongoDB follows an eventual consistency model where updates may not be immediately visible across all systems.

In an event management system, this may result in temporary inconsistencies such as:

1. A ticket still appearing available after it has been booked
2. Recent event updates not immediately visible to all users

While this design improves system performance and fault tolerance, it requires careful handling of critical operations. For example, ticket registration should use transactions or atomic updates to prevent overbooking. Developers must design the system logic to account for these trade-offs. It is to ensure that important operations remain reliable while still benefiting from scalability and flexibility of NoSQL systems.

6.0 Teamwork Roles and Contributions

Name	Contribution
Dheshieghan A/L Saravana Moorthy	• Designed the **NoSQL data model** for the Event Management System • Identified and structured **MongoDB collections** • Developed **JSON document structures** with proper data types • Decided and justified the use of **embedding vs referencing**
Muhammad Adam bin Razali	• Developed CRUD Implementations using insert, query, update and delete operations. • Show example use of query filtering like $lte and $and • Implemented the projections to filter out unwanted details during query • Taking part on completing report including introduction and CRUD Implementation
Pravinraj A/L Sivabathi	• Advanced MongoDB Operations (Indexing & Aggregation): I implemented advanced MongoDB operations by creating both single-field and compound indexes to improve query performance and support efficient data retrieval in the event management system. • Aggregation Pipeline Design: I designed and executed aggregation pipelines to generate analytical insights, including total registrations per event, total revenue per event, and registration status summaries for better decision-making. • Sorting and Query Performance Analysis: I

	demonstrated sorting operations in ascending and descending order and explained how indexing enhances query speed and optimizes database performance.
Muhammad Afiq Danish bin Mohd Hazni	• Developed the Security & Challenges section by discussing NoSQL security concerns • Analyzed real-world risks in NoSQL systems such as unauthorized access, data manipulation and inconsistencies issues • Contributed to the formulation and structuring of the project report documents

7.0 Conclusion

In this project, we built a functional NoSQL backend for an Event Management System using MongoDB. We came up with the design of collections for users, events, and registrations using MongoDB document models. We learned how to use embedding and referencing to store complex data like ticket types effectively.

We also implemented basic CRUD operations to manage our data via the insert, query, update, and delete operations. To make the system faster, we created single-field and compound indexes for common searches. We also used aggregation pipelines, such as total revenue and the number of registrations per event.

Finally, we studied security concerns like access control and injection risks to keep the database safe. This project helped us understand how NoSQL databases provide the flexibility and performance needed for modern applications.

Appendix

Video Presentation Link - https://youtu.be/_RTkglE2Tuc